Politecnico di Milano

Ingegneria del Software – a.a. 2006/07

Appello del 10 luglio 2007

Cognome

Nome

Matricola

Sezione (segnarne una) Baresi ☐, Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 6)

Il seguente metodo statico:

public static int[] fusioneOrdinata (int[ ] a, int[] b) {... }

effettua la fusione ordinata dei due array a e b, ossia fa in modo che, dopo la chiamata, l'array restituito al chiamante contenga tutti e soli gli elementi contenuti, prima della chiamata, nei due array a e b, e inoltre risulti ordinato.

Si scriva una specifica, in JML del metodo fusioneOrdinata nei due seguenti casi:

a- si assume che i due array a e b non abbiano elementi ripetuti, e siano disgiunti (nessun valore int può essere memorizzato in entrambi)

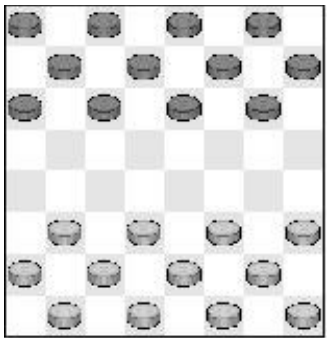
```
//@assignable \nothing
//@requires a!= null && b!= null &&
// ogni valore int memorizzato in a compare una sola volta in a e non compare in b
//@   (\forall int i; 0<=i&&i<a.length; (\num_of int j; 0<=j<a.length; a[j]==a[i])==1 && (\num_of int k; 0<=k<b.length; b[k]==a[i])==0)
// ogni valore int memorizzato in b compare una sola volta in b e non compare in a
//@   (\forall int i; 0<=i&&i<b.length; (\num_of int j; 0<=j<b.length; b[j]==b[i])==1 && (\num_of int k; 0<=k<a.length; a[k]==b[i])==0)
//@ensures  \result != null &&
//\result è ordinato
//@   (\forall int i; 0<=i<\result.length-1; \result [i]<= \result [i+1]) &&
// tutti gli elementi di a stanno in \result
//@          (\forall int i; 0<=i<a.length;(\exists int j; 0<=j<\result.length; a[i]== \result[j])) &&
// tutti gli elementi di b stanno in \result
//@          (\forall int i; 0<=i<b.length;(\exists int j; 0<=j<\result.length; b[i]== \result [j])) &&
// \result non contiene elementi spurii
//@          \result.length = a.length + b.length
```

b- si ammettono anche i casi in cui i due array a e b abbiano elementi ripetuti, e non siano disgiunti

```
//@assignable \nothing
//@requires a!= null && b!= null &&
//@ensures \result != null &&
//\result è ordinato
//@   (\forall int i; 0<=i<\result.length-1; \result[i]<= \result[i+1])  &&
//il numero di volte in cui un qualsiasi numero compare in \result è pari alla somma
// del numero di volte incui compare in a e del numero in cui compare in b
//@  (\forall int i; 0<=i && i<\result.length;
        (\num_of int j; 0<=j<\result.length; \result[j]==\result[i]))  ==
          (\num_of int j; 0<=j<a.length; a[j]== \result[i])+ (\num_of int j; 0<=j<b.length; b[j]== \result[i])
// \result non contiene elementi spurii: ora è diventato pleonastico
//@          \result.length = a.length + b.length
```

Esercizio 2 (punti 12)

Si consideri una versione semplificata del gioco della dama (italiana) in cui le pedine possono solo muovere in avanti. **Scacchiera** e **Pedina** sono le classi principali della soluzione presentata. Il gioco, al quale prendono parte due giocatori, si svolge su una scacchiera formata da 64 caselle: 32 bianche e 32 nere. Le pedine sono 24: 12 bianche e 12 nere e si dispongono sulle caselle scure come indicato in figura.



Le righe e le colonne della scacchiera sono numerate da 1 a 8 dall'alto in basso e da sinistra a destra. La prima pedina nera è quindi in posizione 1:1 (riga:colonna), mentre l'ultima pedina bianca è in posizione 8:8.

La classe **Scacchiera** fornisce i seguenti metodi:

- **public int numPezzi(int colore)** restituisce il numero di pedine sulla scacchiera di un certo colore (0 bianco e 1 nero).
- **public ArrayList<Pedina> pedine(int colore)** restituisce i pezzi sulla scacchiera come una lista di elementi.
- **public ArraList<Pedina> diagonale(Pedina p, char direzione)** restituisce la sequenza di pedine sulla diagonale sinistra (direzione = s, dal basso a dx all'alto a sx) oppure destra (direzione = d, dal basso a sx all'alto a dx) viste dalla pedina p.

La classe Pedina fornisce i seguenti metodi :

- **public int colore()** restituisce il colore della pedina.
- **public int riga()** restituisce la riga in cui la pendina si trova nel momento della chiamata del metodo.
- **public int colonna()** restituisce la colonna in cui la pedina si trova nel momento della chiamata del metodo.

a. Si scriva la post-condizione del metodo pubblico **nuovaPartita()** della classe **Scacchiera**

il metodo numPezzi è ridondante rispetto al metodo size() applicato al risultato di pedine().

```
//@ requires true;

//@ ensures
// presi i due colori
(\forall int i; 0<i<=1;
// il numero di pezzi è uguale a 12
     numPezzi(i) == 12 &&
// tutte le pedine
       (\forall int j; 0<=j<pedine(i).size();
// stanno sulle caselle nere
            pedine(i).get(j).riga()+pedine(i).get(j).colonna()%2==0 &&
// e stanno sulle righe previste
            pedine(i).get(j)colore()==0? 6<= pedine(i).get(j).riga()<=8: 1<= pedine(i).get(j).riga()+<=3))
```

b. Si scriva la pre- e post-condizione del metodo **public bool mossaPossibile(Pedina p, char direzione)** della classe **Scacchiera.** Ogni pedina ha due sole possibilità di movimento di una posizione in diagonale: le pedine bianche si possono muovere di una casella verso l'alto a sinistra (direzione vale s) oppure a destra (direzione vale d), quelle nere verso il basso a sinistra o a destra. La casella di destinazione deve essere libera da pedine. Ad esempio, se una pedina bianca in posizione 6:6 volesse muove in direzione s (sinistra), questo significherebbe spostarla in posizione 5:5 a patto che la casella sia libera. Il metodo ritorna true se la mossa può essere effettuata correttamente, false altrimenti.

```
// se p non esiste o e se non ci sono pedine dell'altro colore, non ha senso porsi il problema
//@ requires p!=null && numPedine(1-p.colore()) > 0 &&
        1<=p.riga()<=8 && 1<=p.colonna()<=8 && 0<=p.colore()<=1 &&
        (direzione == 's' || direzione == 'd')

// ipotizzando che il metodo diagonale() restituisca sempre la diagonale,
// al più con elementi null se la cella è libera
// e che che il primo elemento restituito da diagonale sia
// la pedina stessa

//@ ensures diagonale(p, direzione).size() > 1 &&
            diagonale(p, direzione).get(1)==null ? \result == true: \result == false
```

c. Si scriva la pre- e post-condizione del metodo **void muovi(Pedina p, char direzione)** della classe **Scacchiera**, sapendo la mossa deve essere possibile e l'invocazione del metodo modifica la posizione di p che è anche l'unica pedina che cambia posizione.

```
//@ requires mossaPossibile(p, direzione);

//@ ensures p.colore()==0? (p.riga()==\old(p.riga())-1 &&
                            direzione==’s'? p.colonna()==\old(p.colonna) -1:
                                            p.colonna()==\old(p.colonna) +1):
                             (p.riga()==\old(p.riga())+1 &&
                             direzione==`s'? p.colonna()==\old(p.colonna) +1:
                                            p.colonna()==\old(p.colonna) -1)  &&
// p è l'unica pedina che si muove
   (\forall int i; 0<=i<=1;
        (\forall int j; 0<=j<pedine(i).size(); pedine(i).get(j) != p ->
          pedine(i).get(j).riga() == \old(pedine(i).get(j).riga()) &&
          pedine(i).get(j).colonna() == \old(pedine(i).get(j).colonna())))
```

d. Si vuole specificare la proprietà che il numero complessivo delle pedine sulla scacchiera non è crescente. Di che tipo di proprietà si tratta? Come sarebbe possibile specificarla in JML?

La proprietà è una tipica proprietà evolutiva. JML non offre un costrutto specifico. Occorre quindi aggiungere alla postcondizione di ogni metodo che modifica lo stato della scacchiera una clausola numPezzi(0)<=\old(numPezzi(0)) && numPezzi(1)<=\old(numPezzi(1))

e. Si scriva l'invariante privato della classe **Scacchiera**, ipotizzando il REP indicato qui sotto e ricordando che le pedine possono stare solo sulle caselle nere e che le pedine di ogni colore non possono essere più di 12:

```
    private Pedina[8][8] griglia;

//@ private invariant (\forall int k, 0<=k<=1,
//    (\sum int i; 0<=i<8;
//       (\num_of int j; 0<=j<8; griglia[i][j].colore()==k))<=12) &&

// qui avremmo anche potuto usare anche il metodo pubblico numPezzi()

//    (\forall int i; 0<=i<8;
//       (\forall int j; 0<=j<8; griglia[i][j]!=null->(i+j)%2==1))
```

Esercizio 3 (punti 7)

Si consideri la classe **Poly**, definita a lezione. Si supponga che in essa siano definiti anche i metodi **valuta()** e **soluzioneMinima()**. Il primo metodo calcola il valore float del polinomio corrispondente al parametro. Ad esempio, valuta(3.0), quando è chiamato sul Poly -2x^2+3x+2, restituisce -7. Il secondo metodo restituisce la radice reale minima del Poly, se essa esiste, ossia il più piccolo valore reale x' per cui valuta(x')==0. Per il Poly dell'esempio, soluzioneMinima() restituisce x'= -0.5 (l'altra soluzione è 2.0). La soluzione calcolata è sempre esatta (nel senso che è calcolata fino alla massima precisione possibile con il tipo float). Per il teorema di Ruffini, è noto che se il grado del polinomio supera quattro, la soluzione non può essere calcolata in modo esatto tramite operazioni razionali ed estrazioni di radice. Pertanto, il metodo è definito solo se il grado del polinomio non supera 4.

```
public /*@ pure @*/ class  Poly {
....
//@ensures (*\result è la valutazione del polinomio quando la variabile vale var *);
public float valuta(float var);

//@requires this.deg() <=4;
//@ensures  valuta(\result) ==0 && !(exists float f; f<\result; valuta(f) ==0);
//@signals (NessunaSoluzioneRealeException e) !(exists float f; ; valuta(f) ==0);
public float soluzioneMinima() throws NessunaSoluzioneRealeException
}
```

Un'estensione di Poly calcola la soluzioneMinima anche quando il grado è superiore a 4. In tal caso, dovrà ovviamente ricorrere a tecniche di calcolo numerico. Pertanto, la soluzione sarà approssimata. In particolare, si richiede che la valutazione del risultato differisca dallo zero al più di 0.001.

```
public /*@ pure @*/ class NuovoPoly extends Poly {
//@also
//@requires true;
//@ensures  -0.001<=valuta(\result) <=0.001 &&
//@ !(exists float f; f < \result - 0.001; -0.001<=valuta(f) <=0.001);
public float soluzioneMinima() throws NessunaSoluzioneRealeException;
}
```

La classe NuovoPoly verifica il principio di sostituzione? Mostrare in dettaglio come mai ciò avviene, costruendo le pre e post condizioni complete per il metodo **soluzioneMinima()** in base alle regole del linguaggio JML. La risposta sarà considerata valida solo se correttamente motivata.

La regola delle proprietà è ovviamente verificata (in quanto il metodo soluzioneMinima() è solo un osservatore, e non può quindi influenzare proprietà dello stato astratto).
Apparentemente, la postcondizione di NuovoPoly.soluzioneMinima() è più debole della postcondizione di Poly.soluzioneMinima(). Tuttavia, le regola dei metodi è verificata, perche' la nuova postcondizione si applica solo ai casi non previsti dalla precondizione originale (cioe' solo per this.deg()>4). Per il caso con this.deg()<=4, e' sufficiente che la nuova postcondizione sia non contradditoria con la postcondizione originale.In dettaglio, la regola JML delle specifiche ereditate costruisce la seguente specifica complessiva per il metodo NuovoPoly.soluzioneMinima()

```
//@requires this.deg()<=4 || true
//@ensures  (this.deg()<=4  ==> valuta(\result) ==0 && !(exists float f; f<\result; valuta(f) ==0))
//@            && (true ==> -0.001<=valuta(\result) <=0.001 &&
//@                          !(exists float f; f<\result-0.001; -0.001<=valuta(f) <=0.001);
```

che può essere riscritta come:

```
//@requires true
//@ensures  (this.deg()<=4  ==> valuta(\result) ==0 && !(exists float f; f<\result; valuta(f) ==0)) &&
//@       -0.001<=valuta(\result) <=0.001 && !(exists float f; f<\result-0.001; -0.001<=valuta(f
```

Pertanto, se il grado è minore o uguale a 4, la postcondizione garantisce che la soluzione (se esiste) sia esatta (e quindi a fortiori verifichi anche il vincolo sull'"approssimazione"); negli altri casi, la soluzione sarà solo approssimata. La regola dei metodi e' pertanto verificata.